On an LBF with `L` layers, each using `k` bits per key, the running time for both `insert` and `contains` become `O(L*k)`, but since both `R` and `k` are pre-determined constants, it's still equivalent to `O(1)` and independent on the number of items added to the container.

4.11.4 Compressed Bloom Filter

When used in web caches, the main issue with Bloom filters is not with the RAM they use, but rather, since these filters need to be transferred between proxies, it’s the size of data transmitted on the network.

While at first you might think that’s a moot point, if we plan to compress the Bloom filters before transmitting them through a network, then it does actually have practical relevance: turns out we can optimize the values of the filter’s parameters, which in turn regulate the size of the bit array, and as a result get a larger uncompressed Bloom filter, that could however be compressed more efficiently.

That’s the idea behind compressed bloom filters[24], where the number of hash functions `k` is chosen in such a way that the number of bits with value `1` in the bits array is kept below `m/3`, where `m` is the size of the array. As a consequence, at least `2/3` of the buffer’s bits will always be set to `0`, and we can take advantage of this fact to compress the bit array more efficiently.

Each proxy would then have to decompress the Bloom filter before looking up elements. Clearly now we have another target to optimize: we need to find a compromise between